Лекция 10: монады

Функциональное программирование на Haskell

Алексей Романов

17 октября 2026 г.

МИЭТ

Монады

- Монады расширяют возможности аппликативных функторов так же, как те расширяют возможности функторов.
- А именно, там добавляется

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  (>>) :: m a -> m b -> m b
  mx >> my =
```

Монады

- Монады расширяют возможности аппликативных функторов так же, как те расширяют возможности функторов.
- А именно, там добавляется

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  (>>) :: m a -> m b -> m b
  mx >> my = mx >>=
```

Монады

- Монады расширяют возможности аппликативных функторов так же, как те расширяют возможности функторов.
- А именно, там добавляется

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  (>>) :: m a -> m b -> m b
  mx >> my = mx >>= \_ -> my
```

Монады

- Монады расширяют возможности аппликативных функторов так же, как те расширяют возможности функторов.
- А именно, там добавляется

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  (>>) :: m a -> m b -> m b
  mx >> my = mx >>= \_ -> my
```

- Для законных экземпляров `mx >> my` = `mx *> my`.
- Историческое имя `pure` — `return`. `return` не завершает функцию или `do`-блок.

От функторов к монадам

- Больше похоже на знакомые типы, если поменять аргументы `(>>=)` местами:

```
fmap  ::   (a -> b) -> f a -> f b
(<*>) :: f (a -> b) -> f a -> f b
(=<<) :: (a -> m b) -> m a -> m b
```

Что можно сделать с монадами

- В прошлой лекции: *«Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.»*
- Монады снимают это ограничение. Теперь можно написать

```
ifM :: Monad m => m Bool -> m a -> m a -> m a
ifM mCond mThen mElse =
  mCond >>=
```

Что можно сделать с монадами

- В прошлой лекции: *«Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.»*
- Монады снимают это ограничение. Теперь можно написать

```
ifM :: Monad m => m Bool -> m a -> m a -> m a
ifM mCond mThen mElse =
  mCond >>= \cond ->
```

Что можно сделать с монадами

- В прошлой лекции: *«Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.»*
- Монады снимают это ограничение. Теперь можно написать

```
ifM :: Monad m => m Bool -> m a -> m a -> m a
ifM mCond mThen mElse =
  mCond >>= \cond ->
    if cond then mThen else mElse
ghci> ifM [True] [1] [2,3]
```

Что можно сделать с монадами

- В прошлой лекции: *«Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.»*
- Монады снимают это ограничение. Теперь можно написать

```
ifM :: Monad m => m Bool -> m a -> m a -> m a
ifM mCond mThen mElse =
  mCond >>= \cond ->
    if cond then mThen else mElse
ghci> ifM [True] [1] [2,3]
[1]
ghci> ifM [False] [1] [2,3]
```

Что можно сделать с монадами

- В прошлой лекции: *«Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.»*
- Монады снимают это ограничение. Теперь можно написать

```
ifM :: Monad m => m Bool -> m a -> m a -> m a
ifM mCond mThen mElse =
  mCond >>= \cond ->
    if cond then mThen else mElse
ghci> ifM [True] [1] [2,3]
[1]
ghci> ifM [False] [1] [2,3]
[2,3]
ghci> ifM [True, False] [1] [2,3]
```

Что можно сделать с монадами

- В прошлой лекции: *«Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.»*
- Монады снимают это ограничение. Теперь можно написать

```
ifM :: Monad m => m Bool -> m a -> m a -> m a
ifM mCond mThen mElse =
  mCond >>= \cond ->
    if cond then mThen else mElse
ghci> ifM [True] [1] [2,3]
[1]
ghci> ifM [False] [1] [2,3]
[2,3]
ghci> ifM [True, False] [1] [2,3]
[1,2,3]
```

Стрелки Клейсли

- Тип второго аргумента `(>>=)`, `a -> m b`, где `m` — монада, называется *стрелкой Клейсли*.
- `a` и `b` могут быть как переменными типа, так и конкретными типами.
- Стрелки Клейсли можно композировать (импорт `Control.Monad`):

```
(>=>) :: Monad m => (a -> m b) -> (b -> m c) ->
                    (a -> m c)
f >=> g = \x ->
```

Стрелки Клейсли

- Тип второго аргумента `(>>=)`, `a -> m b`, где `m` — монада, называется *стрелкой Клейсли*.
- `a` и `b` могут быть как переменными типа, так и конкретными типами.
- Стрелки Клейсли можно композировать (импорт `Control.Monad`):

```
(>=>) :: Monad m => (a -> m b) -> (b -> m c) ->
                    (a -> m c)
f >=> g = \x -> f x >>=
```

Стрелки Клейсли

- Тип второго аргумента `(>>=)`, `a -> m b`, где `m` — монада, называется *стрелкой Клейсли*.
- `a` и `b` могут быть как переменными типа, так и конкретными типами.
- Стрелки Клейсли можно композировать (импорт `Control.Monad`):

```
(>=>) :: Monad m => (a -> m b) -> (b -> m c) ->
                    (a -> m c)
f >=> g = \x -> f x >>= g
```

- Есть и аналоги `filter`, `map` и т.д., работающие со стрелками Клейсли. Их можно найти в документации `Control.Monad`.

Законы монад

- Левая единица:

```
pure x >>= f = f x
```

- Правая единица:

```
mx >>= pure = mx
```

- Ассоциативность:

```
(mx >>= f) >>= g = mx >>= (\x -> f x >>= g)
```

- Кроме этого, должно быть согласование с <*>:

```
mf <*> mx =
  mf >>= (\f -> mx >>= (\x -> pure (f x)))
```

  Удобная запись немного позже.

- Может быть, они не совсем интуитивны.

Законы монад: композиция

- С помощью монадической композиции они записываются естественнее:
- `pure >=> f = f` (левая единица)
- `f >=> pure = f` (правая единица)
- `(f >=> g) >=> h = f >=> (g >=> h)` (ассоциативность)

Примеры монад

- `Maybe` — монада. Определим:

```
instance Monad Maybe where
  -- (>>=) :: Maybe a -> (a -> Maybe b) ->
  --          Maybe b
  Nothing >>= _ =
```

Примеры монад

- `Maybe` — монада. Определим:

```
instance Monad Maybe where
  -- (>>=) :: Maybe a -> (a -> Maybe b) ->
  --          Maybe b
  Nothing >>= _ = Nothing
  Just x  >>= f =
```

Примеры монад

- `Maybe` — монада. Определим:

```
instance Monad Maybe where
  -- (>>=) :: Maybe a -> (a -> Maybe b) ->
  --          Maybe b
  Nothing >>= _ = Nothing
  Just x  >>= f = f x
```

Монада списков

- Списки тоже монада:

```
-- (>>=) ::
```

Монада списков

- Списки тоже монада:

```
-- (>>=) :: [a] -> (a -> [b]) -> [b]
[]     >>= _ =
```

Монада списков

- Списки тоже монада:

```
-- (>>=) :: [a] -> (a -> [b]) -> [b]
[]     >>= _ = []
(x:xs) >>= f =
```

Монада списков

- Списки тоже монада:

```
-- (>>=) :: [a] -> (a -> [b]) -> [b]
[]     >>= _ = []
(x:xs) >>= f = f x ++ (xs >>= f)
```

Или проще:

```
xs >>= f = [y |
```

Монада списков

- Списки тоже монада:

```
-- (>>=) :: [a] -> (a -> [b]) -> [b]
[]     >>= _ = []
(x:xs) >>= f = f x ++ (xs >>= f)
```

Или проще:

```
xs >>= f = [y | x <- xs, y <- f x]
```

Ещё примеры монад

- `instance Monad Identity`, где `newtype Identity a = Identity a`.
- `instance Monad (Either c)`.
- `instance Monad ((->) c)`: функции с общим окружением типа `c` (модель `Reader`).
- `instance Monoid c => Monad ((,) c)`: пары `(c, a)` с накоплением моноида `c` (модель `Writer`). У библиотечного `Writer c a` порядок компонентов — `(a, c)`.

Монады в C++: std::optional

- В C++23 у std::optional есть and_then — аналог >>=:

```
std::optional<int> parseInt(std::string_view);
std::optional<double> reciprocal(int);

auto result = parseInt("4").and_then(reciprocal);
```

Монады в C++: `std::optional`

- В C++23 у `std::optional` есть `and_then` — аналог `>>=`:

```
std::optional<int> parseInt(std::string_view);
std::optional<double> reciprocal(int);

auto result = parseInt("4").and_then(reciprocal);
```

- Соответствующее выражение на Haskell:

```
result = parseInt "4" >>= reciprocal
```

- Если `parseInt` вернул `std::nullopt`, `reciprocal` не вызывается.
- У `std::expected<T, E>` в C++23 тоже есть `and_then`: аналог связывания для `Either e`, сохраняющий ошибку.
- В других языках эта операция часто называется `flatMap`.

...и не монад

- Есть и примеры аппликативных функторов, для которых нельзя определить экземпляр монады, удовлетворяющий законам:
- ```
  newtype ConstInt a = ConstInt Int
  fmap f (ConstInt x) = ConstInt x
  pure _ = ConstInt 0
  ConstInt x <*> ConstInt y = ConstInt (x + y)
  ```
- Легко увидеть, что `pure x >>= f = f x` не может выполняться ни для какого определения `>>=`:
  ```

...и не монад

- Есть и примеры аппликативных функторов, для которых нельзя определить экземпляр монады, удовлетворяющий законам:
- ```
  newtype ConstInt a = ConstInt Int
  fmap f (ConstInt x) = ConstInt x
  pure _ = ConstInt 0
  ConstInt x <*> ConstInt y = ConstInt (x + y)
  ```
- Легко увидеть, что `pure x >>= f = f x` не может выполняться ни для какого определения `>>=`: правая часть зависит от `x`, а левая нет.
  ```

...и не монад

- Есть и примеры аппликативных функторов, для которых нельзя определить экземпляр монады, удовлетворяющий законам:
- ```
  newtype ConstInt a = ConstInt Int
  fmap f (ConstInt x) = ConstInt x
  pure _ = ConstInt 0
  ConstInt x <*> ConstInt y = ConstInt (x + y)
  ```
- Легко увидеть, что `pure x >>= f = f x` не может выполняться ни для какого определения `>>=`: правая часть зависит от `x`, а левая нет.
- Ещё пример — `ZipList` (если не допускать *только* бесконечные списки, как в домашнем задании, или активное использование $\bot$).
  ```

do-нотация

- Для монад есть специальный синтаксис, которым часто удобнее пользоваться.
- Скажем, у нас есть цепочка операций

```
action1 >>= (\x1 -> action2 x1 >>=
  (\x2 -> action3 x1 x2 >> action4 x1))
```

- Сначала перепишем так:

```
action1 >>= \x1 ->
  action2 x1 >>= \x2 ->
  action3 x1 x2 >>
  action4 x1
```

- В do-блоке строки вида `action1 >>= \x1 ->` превращаются в `x1 <- action1`, а `>>` пропадает:

```
do x1 <- action1
   x2 <- action2 x1
   action3 x1 x2
   action4 x1
```

Законы монад в do-нотации

- Законы монад также можно записать через do:
- ```
  do y <- pure x        =
     f y
  ```
  ```

Законы монад в do-нотации

- Законы монад также можно записать через do:
- ```
  do y <- pure x          =  f x
     f y
  ```
- ```
  do x <- mx              =
     pure x
  ```

Законы монад в do-нотации

- Законы монад также можно записать через do:
- ```
  do y <- pure x          =  f x
     f y
  ```
- ```
  do x <- mx              =  mx
     pure x
  ```
- ```
  do y <- do x <- mx      =
             f x
     g y
  ```
  ```

Законы монад в do-нотации

- Законы монад также можно записать через do:
- ```
  do y <- pure x          =  f x
     f y
  ```
- ```
  do x <- mx              =  mx
     pure x
  ```
- ```
  do y <- do x <- mx      =  do x <- mx
             f x                 y <- f x
     g y                         g y
  ```
- Согласование с <*> (не закон монады):

  ```
  mf <*> mx               =
  ```
  ```

Законы монад в do-нотации

- Законы монад также можно записать через do:
- ```
  do y <- pure x          =  f x
     f y
  ```
- ```
  do x <- mx              =  mx
     pure x
  ```
- ```
  do y <- do x <- mx      =  do x <- mx
             f x                 y <- f x
     g y                         g y
  ```
- Согласование с <*> (не закон монады):

  ```
  mf <*> mx               =  do f <- mf
                                x <- mx
                                pure (f x)
  ```
  ```

Общая форма do-нотации

- Каждая строка do-блока имеет вид `образец <- м_выражение`, `let образец = выражение` или просто `м_выражение`.
- Первые два вида не могут быть в конце.
- `м_выражение` должно иметь тип `m a` для какой-то монады `m` и типа `a`.
- `m` одна для всех строк, `a` могут различаться.
- `образец` в строке с `<-` имеет тип `a`.
- Если образец после `<-` может не совпасть, нужен экземпляр `MonadFail`; при несовпадении вызывается `fail`.

Несовпадение образца в do-блоке

- Для Maybe несовпадение образца даёт Nothing:

```haskell
safeHead :: Maybe [Int] -> Maybe Int
safeHead mx = do
  (x : _) <- mx
  pure x
```

Несовпадение образца в do-блоке

- Для Maybe несовпадение образца даёт Nothing:

```
safeHead :: Maybe [Int] -> Maybe Int
safeHead mx = do
  (x : _) <- mx
  pure x
ghci> safeHead (Just [2, 3])
```

Несовпадение образца в do-блоке

- Для Maybe несовпадение образца даёт Nothing:

```
safeHead :: Maybe [Int] -> Maybe Int
safeHead mx = do
  (x : _) <- mx
  pure x
ghci> safeHead (Just [2, 3])
Just 2
ghci> safeHead (Just [])
```

Несовпадение образца в do-блоке

- Для Maybe несовпадение образца даёт Nothing:

```
safeHead :: Maybe [Int] -> Maybe Int
safeHead mx = do
  (x : _) <- mx
  pure x
ghci> safeHead (Just [2, 3])
Just 2
ghci> safeHead (Just [])
Nothing
```

- При несовпадении pure x не выполняется.
- Подробности — в документации MonadFail.
- Дальше нам достаточно образцов, которые всегда совпадают.

Функции из `Control.Monad`

- Нужен импорт `Control.Monad`: здесь определены `(>=>)`, `join`, `zipWithM` и `foldM`.

Функции из `Control.Monad`

- Нужен импорт `Control.Monad`: здесь определены `(>=>)`, `join`, `zipWithM` и `foldM`.
- `join :: Monad m => m (m a) -> m a`

Функции из `Control.Monad`

- Нужен импорт `Control.Monad`: здесь определены `(>=>)`, `join`, `zipWithM` и `foldM`.
- `join :: Monad m => m (m a) -> m a`
- ```
  zipWithM :: Applicative m =>
    (a -> b -> m c) -> [a] -> [b] -> m [c]
  ```
  ```

Функции из `Control.Monad`

- Нужен импорт `Control.Monad`: здесь определены `(>=>)`, `join`, `zipWithM` и `foldM`.
- `join :: Monad m => m (m a) -> m a`
- ```
  zipWithM :: Applicative m =>
    (a -> b -> m c) -> [a] -> [b] -> m [c]
  ```
- ```
  foldM :: (Foldable t, Monad m) =>
    (b -> a -> m b) -> b -> t a -> m b
  ```

  Свёртка слева: шаг возвращает результат в монаде.

Функции из `Control.Monad`

- Нужен импорт `Control.Monad`: здесь определены `(>=>)`, `join`, `zipWithM` и `foldM`.
- `join :: Monad m => m (m a) -> m a`
- ```
  zipWithM :: Applicative m =>
    (a -> b -> m c) -> [a] -> [b] -> m [c]
  ```
- ```
  foldM :: (Foldable t, Monad m) =>
    (b -> a -> m b) -> b -> t a -> m b
  ```

  Свёртка слева: шаг возвращает результат в монаде.
- Подумайте, как эти функции работают для `m = []` и `m = Maybe`.

`foldM`: пример

- Проводим операции по порядку. Отрицательный промежуточный баланс означает отказ:

```
step :: Integer -> Integer -> Maybe Integer
step balance delta
  | balance + delta < 0 = Nothing
  | otherwise = Just (balance + delta)
```

- Начальный баланс — `10`:

```
ghci> foldM step 10 [-3, 5]
```

foldM: пример

- Проводим операции по порядку. Отрицательный промежуточный баланс означает отказ:

```
step :: Integer -> Integer -> Maybe Integer
step balance delta
  | balance + delta < 0 = Nothing
  | otherwise = Just (balance + delta)
```

- Начальный баланс — 10:

```
ghci> foldM step 10 [-3, 5]
Just 12
ghci> foldM step 10 [-12, 5]
```

foldM: пример

- Проводим операции по порядку. Отрицательный промежуточный баланс означает отказ:

```
step :: Integer -> Integer -> Maybe Integer
step balance delta
  | balance + delta < 0 = Nothing
  | otherwise = Just (balance + delta)
```

- Начальный баланс — 10:

```
ghci> foldM step 10 [-3, 5]
Just 12
ghci> foldM step 10 [-12, 5]
Nothing
```

- Во втором случае +5 уже не обрабатывается, хотя итоговая сумма могла бы стать положительной.

join и >>=: упражнение

- Выразите `join` через `>>=`, а затем `>>=` через `fmap` и `join`:

```
join :: Monad m => m (m a) -> m a
join mm = ???

mx >>= g = ???
```

- Подсказка: подумайте о типе `fmap g mx`.

Отбор значений: guard

- `guard` из `Control.Monad` проверяет условие:

```
import Control.Monad (guard)
guard :: Alternative f => Bool -> f ()
-- Для Maybe:
guard True  =
```

Отбор значений: guard

- `guard` из `Control.Monad` проверяет условие:

```
import Control.Monad (guard)
guard :: Alternative f => Bool -> f ()
-- Для Maybe:
guard True  = Just ()
guard False =
```

Отбор значений: guard

- `guard` из `Control.Monad` проверяет условие:

```
import Control.Monad (guard)
guard :: Alternative f => Bool -> f ()
-- Для Maybe:
guard True  = Just ()
guard False = Nothing
```

- Можно продолжить вычисление, только если число чётное:

```
evenSquare :: Maybe Int -> Maybe Int
evenSquare mx = do
  x <- mx
  guard (even x)
  pure (x * x)
```

- У `Maybe` и списков есть `Alternative`; для списков `guard False` даёт `[]`.

Монадические включения

- `MonadComprehensions` обобщает синтаксис списковых включений на монады:

```
{-# LANGUAGE MonadComprehensions #-}
evenSquare mx = [x * x | x <- mx, even x]
```

Монадические включения

- `MonadComprehensions` обобщает синтаксис списковых включений на монады:

```
{-# LANGUAGE MonadComprehensions #-}
evenSquare mx = [x * x | x <- mx, even x]
```

- Перевод в `do`-нотацию:

```
evenSquare mx = do x <- mx
                   guard (even x)
                   return (x * x)
```

Монадические включения

- `MonadComprehensions` обобщает синтаксис списковых включений на монады:

```
{-# LANGUAGE MonadComprehensions #-}
evenSquare mx = [x * x | x <- mx, even x]
```

- Перевод в `do`-нотацию:

```
evenSquare mx = do x <- mx
                   guard (even x)
                   return (x * x)
```

- Например, для `Maybe`:

```
ghci> :set -XMonadComprehensions
ghci> [x * x | x <- Just 2, even x]
```

Монадические включения

- `MonadComprehensions` обобщает синтаксис списковых включений на монады:

```
{-# LANGUAGE MonadComprehensions #-}
evenSquare mx = [x * x | x <- mx, even x]
```

- Перевод в `do`-нотацию:

```
evenSquare mx = do x <- mx
                   guard (even x)
                   return (x * x)
```

- Например, для `Maybe`:

```
ghci> :set -XMonadComprehensions
ghci> [x * x | x <- Just 2, even x]
Just 4
ghci> [x * x | x <- Just 3, even x]
```

Монадические включения

- `MonadComprehensions` обобщает синтаксис списковых включений на монады:

```
{-# LANGUAGE MonadComprehensions #-}
evenSquare mx = [x * x | x <- mx, even x]
```

- Перевод в `do`-нотацию:

```
evenSquare mx = do x <- mx
                   guard (even x)
                   return (x * x)
```

- Например, для `Maybe`:

```
ghci> :set -XMonadComprehensions
ghci> [x * x | x <- Just 2, even x]
Just 4
ghci> [x * x | x <- Just 3, even x]
Nothing
```

Монада State

- Рассмотрим ещё один, более сложный пример:
- `newtype State s a = State { runState :: s -> (a, s) }`

  Это «вычисления с состоянием», которые могут выдать результат, зависящий от состояния, и изменить это состояние.
- `runState` принимает вычисление и начальное состояние, возвращает результат и конечное состояние.

State: экземпляры Functor и Applicative

- Как сделать эти вычисления аппликативным функтором?

```
instance Functor (State s) where
  fmap f (State gx) = State (\s1 ->
```

State: экземпляры Functor и Applicative

- Как сделать эти вычисления аппликативным функтором?

```
instance Functor (State s) where
  fmap f (State gx) = State (\s1 ->
    let (x, s2) = gx s1
    in
```

State: экземпляры Functor и Applicative

- Как сделать эти вычисления аппликативным функтором?

```
instance Functor (State s) where
  fmap f (State gx) = State (\s1 ->
    let (x, s2) = gx s1
    in (f x, s2))

instance Applicative (State s) where
  pure x = State (\s1 ->
```

State: экземпляры Functor и Applicative

- Как сделать эти вычисления аппликативным функтором?

```
instance Functor (State s) where
  fmap f (State gx) = State (\s1 ->
    let (x, s2) = gx s1
    in (f x, s2))

instance Applicative (State s) where
  pure x = State (\s1 -> (x, s1))
  (State gf) <*> (State gx) = State (\s1 ->
    let (f, s2) = gf s1
```

State: экземпляры Functor и Applicative

- Как сделать эти вычисления аппликативным функтором?

```
instance Functor (State s) where
  fmap f (State gx) = State (\s1 ->
    let (x, s2) = gx s1
    in (f x, s2))

instance Applicative (State s) where
  pure x = State (\s1 -> (x, s1))
  (State gf) <*> (State gx) = State (\s1 ->
    let (f, s2) = gf s1
        (x, s3) = gx s2
    in
```

State: экземпляры Functor и Applicative

- Как сделать эти вычисления аппликативным функтором?

```
instance Functor (State s) where
  fmap f (State gx) = State (\s1 ->
    let (x, s2) = gx s1
    in (f x, s2))

instance Applicative (State s) where
  pure x = State (\s1 -> (x, s1))
  (State gf) <*> (State gx) = State (\s1 ->
    let (f, s2) = gf s1
        (x, s3) = gx s2
    in (f x, s3))
```

Монада State

- А теперь монадой:

```
instance Monad (State s) where
  State gx >>= f = State (\s1 ->
    let (x, s2) = gx s1
```

Монада State

- А теперь монадой:

```
instance Monad (State s) where
  State gx >>= f = State (\s1 ->
    let (x, s2) = gx s1
        State gy = f x
    in
```

Монада State

- А теперь монадой:

```
instance Monad (State s) where
  State gx >>= f = State (\s1 ->
    let (x, s2) = gx s1
        State gy = f x
    in gy s2)
```

Монада State

- А теперь монадой:

```
instance Monad (State s) where
  State gx >>= f = State (\s1 ->
    let (x, s2) = gx s1
        State gy = f x
    in gy s2)
```

- Определим вспомогательные функции:

```
get :: State s s
get = State
```

Монада State

- А теперь монадой:

```
instance Monad (State s) where
  State gx >>= f = State (\s1 ->
    let (x, s2) = gx s1
        State gy = f x
    in gy s2)
```

- Определим вспомогательные функции:

```
get :: State s s
get = State (\s -> (s, s))

put :: s -> State s ()
put x = State
```

Монада State

- А теперь монадой:

```
instance Monad (State s) where
  State gx >>= f = State (\s1 ->
    let (x, s2) = gx s1
        State gy = f x
    in gy s2)
```

- Определим вспомогательные функции:

```
get :: State s s
get = State (\s -> (s, s))

put :: s -> State s ()
put x = State (\_ -> ((), x))

modify :: (s -> s) -> State s ()
modify f = do x <- get
              put (f x)
```

State: пример вычисления

- Прочитаем состояние, увеличим его и вернём оба значения:

```haskell
increment :: State Int (Int, Int)
increment = do
  old <- get
  modify (+1)
  new <- get
  pure (old, new)
```

State: пример вычисления

- Прочитаем состояние, увеличим его и вернём оба значения:

```
increment :: State Int (Int, Int)
increment = do
  old <- get
  modify (+1)
  new <- get
  pure (old, new)
```

- Запустим вычисление с начальным состоянием `10`:

```
ghci> runState increment 10
((10,11),11)
```

- `(10,11)` — результат, последнее `11` — конечное состояние.

State и traverse: упражнение

- Дополнительное упражнение: пронумеруйте элементы списка с помощью состояния.
- Каждый элемент получает текущий номер, затем счётчик увеличивается:

```
number :: a -> State Int (Int, a)
number x = ???
```

- Подсказка: используйте `get`, `modify` и `pure`.

State и traverse: упражнение

- Дополнительное упражнение: пронумеруйте элементы списка с помощью состояния.
- Каждый элемент получает текущий номер, затем счётчик увеличивается:

```
number :: a -> State Int (Int, a)
number x = ???
```

- Подсказка: используйте `get`, `modify` и `pure`.
- Ожидаемый результат обхода:

```
ghci> runState (traverse number ["a", "b"]) 0
([(0,"a"),(1,"b")],2)
```

Монада `State` в библиотеках

- transformers содержит сами трансформеры монад (`StateT`, `ReaderT`, ...; о них позже в курсе), модуль `Control.Monad.Trans.State`.
- mtl строится на transformers и добавляет классы вроде `MonadState`, чтобы `get` и `put` работали без явного `lift`; модуль `Control.Monad.State`.
- В обоих `State s` определён как `StateT s Identity`.
- Поэтому *конструктора* `State` там нет: вместо него функция
  ```
  state :: (s -> (a, s)) -> State s a
  ```
- `runState`, `get`, `put`, `modify` есть, а также `evalState` (только результат) и `execState` (только состояние).

Дополнительное чтение

- Functors, Applicatives, and Monads
- Typeclassopedia (ещё раз)
- What I Wish I Knew When Learning Haskell: Monads
- All About Monads
- И более сложные монады:
  - The Mother of all Monads
  - Mindfuck: The Reverse State Monad
  - The Curious Time-Traveling Reverse State Monad